Lộ trình dự báo doanh thu từ tuần sang ngày sẵn sàng triển khai

Tóm tắt điều hành

Thiết kế vững chắc nhất cho bài toán này là mô hình phân cấp thời gian theo hướng **top-down**: trước hết dự báo **doanh thu danh nghĩa theo tuần**, sau đó phân bổ dự báo đó xuống **doanh thu danh nghĩa theo ngày**, tiếp theo có thể học thêm phần **sai số dư hằng ngày** nếu cần, và cuối cùng thực hiện **reconcile** để bảo đảm tổng doanh thu theo ngày cộng lại đúng bằng tổng doanh thu tuần.

Đây là cách sạch nhất để giữ tính nhất quán giữa các cấp thời gian, tránh để mô hình chính bị nhiễu bởi dao động nhu cầu hằng ngày, nhưng vẫn tạo được chuỗi dự báo theo ngày cho giai đoạn 2023–2024. Các phương pháp dự báo phân cấp được thiết kế đúng cho yêu cầu "coherence" này, còn phương pháp top-down hoạt động bằng cách dự báo ở cấp tổng hợp trước rồi phân rã xuống cấp chi tiết hơn.

Phương án triển khai an toàn nhất gồm 5 tầng:

1. **Level 0:** xây dựng daily mart và weekly mart an toàn chống leakage từ các file được cung cấp.
2. **Level 1:** huấn luyện mô hình dự báo tuần bằng các đặc trưng lag/rolling nội bộ và các đặc trưng lịch biết trước.
3. **Level 2:** phân bổ doanh thu tuần xuống ngày bằng mô hình trọng số softmax.
4. **Level 3:** tùy chọn huấn luyện mô hình residual chỉ trên dự báo daily OOF nền.
5. **Level 4:** reconcile và chuẩn hóa an toàn để dự báo không âm và tổng ngày bằng đúng tổng tuần.

Nên dùng **Polars lazy query** cho ETL và feature engineering. Lazy API của Polars phù hợp vì nó tối ưu toàn bộ query plan, hỗ trợ streaming cho dữ liệu lớn hơn bộ nhớ, và dynamic weekly window có thể bắt đầu từ thứ Hai, phù hợp với yêu cầu ISO week.

Khuyến nghị tổng quát:

- Không dùng một mô hình direct daily duy nhất làm hệ thống chính.
- Không dùng CPI, lockdown hoặc dữ liệu macro bên ngoài.
- Giữ target là **doanh thu danh nghĩa**.
- Ghi nhận xu hướng "giống lạm phát" bằng tín hiệu nội bộ như AOV, COGS-per-order, YoY reference và time index.
- Chỉ dùng COVID smoothing cho bản sao đặc trưng dài hạn, không smoothing trực tiếp target.
- Nếu sau này cần dự báo thêm COGS, có thể tái sử dụng Level 0 và cấu trúc phân cấp thời gian, rồi huấn luyện song song stack COGS hoặc mô hình margin-ratio trên stack revenue.

Kiến trúc khuyến nghị

Mô hình phân cấp từ tuần sang ngày là mặc định phù hợp vì tín hiệu kinh doanh chính thường ổn định ở tần suất tuần hơn là ngày. Nhu cầu, conversion, giá trị giỏ hàng, ràng buộc tồn kho, áp lực trả hàng và cơ

cấu khách hàng đều ổn định hơn khi nhìn theo tuần. Tuy nhiên, đầu ra yêu cầu là theo ngày, nên cần một tầng thứ hai để học hình dạng phân bổ trong tuần, thay vì bắt một mô hình duy nhất học cả cấu trúc kinh doanh dài hạn lẫn dao động nhỏ hằng ngày.

Lý do thứ hai là ràng buộc chống leakage. Khi dự báo cho 2023–2024, ta không biết trước session tương lai, đơn hàng tương lai, stockout tương lai, return tương lai hay review tương lai, trừ khi dự báo riêng chúng. Vì vậy, weekly core chỉ nên dùng các đặc trưng quá khứ dạng lag/rolling hoặc các đặc trưng thật sự biết trước như biến lịch, ngày trong tuần, tháng, tuần ISO, hoặc lịch khuyến mãi nội bộ đã được biết trước.

```
flowchart LR
    A[Provided files] --> B[Level 0 daily marts]
    B --> C[Weekly feature mart]
    C --> D[Level 1 weekly revenue model]
    D --> E[Weekly revenue forecast]

    B --> F[Level 2 daily allocation model]
    E --> G[Base daily forecast]
    F --> G

    G --> H[Level 3 residual correction]
    H --> I[Adjusted daily forecast]

    E --> J[Level 4 safe reconciliation]
    I --> J

    J --> K[Final daily nominal revenue]
```

Quy tắc sản xuất cuối cùng:

$$\text{Final Daily Revenue} = \text{Reconcile}\left(\text{Weekly Forecast} \times \text{Daily Weight} \times e^{\text{Residual Correction}}\right)$$

Thiết kế này bảo đảm bốn điều:

$$\text{Final Daily Revenue} \geq 0$$

$$\sum_{d \in \text{week}} \text{Final Daily Revenue}_d = \text{Weekly Revenue Forecast}$$

Và toàn bộ inference path chỉ dùng dữ liệu đã quan sát trong quá khứ hoặc dữ liệu biết trước tại thời điểm dự báo.

Một edge case cần xử lý ngay từ đầu là **partial ISO week** ở ranh giới forecast. ISO week bắt đầu vào thứ Hai và kết thúc vào Chủ nhật. Ranh giới ISO year không trùng hoàn toàn với năm dương lịch. Vì vậy, một số ngày đầu hoặc cuối năm dương lịch có thể thuộc ISO year trước hoặc sau.

Kế hoạch sạch nhất:

- Huấn luyện weekly model chỉ trên các ISO week đầy đủ.
- Dự báo các ISO week tương lai đầy đủ.
- Với các ngày ranh giới nằm ngoài tuần đầy đủ, dùng một daily boundary shim nhỏ chỉ dựa trên date features và trọng số weekday lịch sử.

Cách này tránh làm bẩn weekly mart bằng target của các tuần không đầy đủ.

Nền tảng dữ liệu vững chắc

Tầng data engineering quan trọng hơn lựa chọn mô hình. Polars phù hợp vì lazy API cho phép scan file, tối ưu join, đẩy filter/projection xuống thấp hơn, và dùng streaming khi cần. Ngoài ra, dynamic time grouping của Polars không tự tạo các window rỗng, vì vậy nên tạo một **daily spine** hoàn chỉnh trước, sau đó left join từng block mart vào daily spine, rồi mới aggregate lên cấp tuần.

Thiết kế mart chuẩn

Trước tiên xây dựng một daily mart cho từng block, sau đó join tất cả vào một bảng `daily_spine(date)` duy nhất:

- `sales_daily` từ `sales.csv`
- `demand_daily` từ `web_traffic.csv`
- `orders_daily` từ `orders.csv` và `order_items.csv`
- `basket_daily` từ `order_items.csv`, `products.csv`, `promotions.csv`
- `supply_daily` từ `inventory.csv`, `shipments.csv`
- `leakage_daily` từ `returns.csv`, `reviews.csv`
- `customer_daily` từ `customers.csv` và `orders.csv`
- `calendar_daily` từ chính trường ngày

Không join trực tiếp các bảng giao dịch raw với nhau trước. Hãy aggregate từng nguồn về một dòng trên mỗi ngày, rồi mới join các daily mart. Cách này tránh row explosion và tránh nhân đôi measure một cách âm thầm.

Mapping ISO week

ISO week là backbone phù hợp. Theo chuẩn ISO, tuần bắt đầu vào thứ Hai và kết thúc vào Chủ nhật; tuần ISO đầu tiên là tuần có chứa ngày thứ Năm đầu tiên của năm. Polars hỗ trợ logic này qua `dt.iso_year()` và `dt.week()`.

Các trường lịch chuẩn nên dùng ở mọi nơi:

- `date`
- `iso_year`
- `iso_week`
- `iso_weekday`

- `week_id = "{iso_year}-W{iso_week:02d}"`
- `week_start`: ngày thứ Hai đại diện cho ISO week
- `month`, `quarter`, `day_of_month`, `day_of_year`
- `time_index_day`
- `time_index_week`

Chi tiết quan trọng: không chỉ dựa vào `lag_52w` cho seasonality theo năm, vì một số ISO year có 53 tuần. Thay vào đó, nên tạo reference "same ISO week last year" bằng cách self-join trên `(iso_year - 1, iso_week)`, sau đó chỉ dùng `lag_52w` hoặc `lag_53w` làm fallback. Cách này bền vững hơn so với shift mù 52 tuần.

Công thức lõi

Dùng một epsilon thống nhất:

$$\varepsilon = 10^{-9}$$

Safe division:

$$\text{safe_div}(a, b) = \frac{a}{\max(|b|, \varepsilon)}$$

Weighted mean:

$$\text{weighted_mean}(x, w) = \frac{\sum_i x_i w_i}{\max(\sum_i w_i, \varepsilon)}$$

Traffic-source entropy:

$$\text{share}_{s,w} = \frac{\text{sessions}_{s,w}}{\max(\text{sessions}_w, \varepsilon)}$$

$$\text{entropy}_w = -\sum_s \text{share}_{s,w} \log(\text{share}_{s,w} + \varepsilon)$$

Past-only rolling features dùng để dự báo tuần t:

$$x_{\text{lag1}}(t) = x_{t-1}$$

$$x_{\text{lag4}}(t) = x_{t-4}$$

$$x_{\text{ma4}}(t) = \frac{x_{t-4} + x_{t-3} + x_{t-2} + x_{t-1}}{4}$$

$$x_{\text{vol4}}(t) = \text{std}(x_{t-4}, x_{t-3}, x_{t-2}, x_{t-1})$$

$$x_{\text{growth1}}(t) = \text{clip}\left(\frac{x_{t-1} - x_{t-2}}{\max(|x_{t-2}|, \varepsilon)}, -3, 3\right)$$

Trong Polars, rolling window mặc định có thể bao gồm dòng hiện tại, còn `shift()` tương đương trực tiếp với LAG. Vì vậy, mọi rolling feature chống leakage nên được xây dựng theo dạng:

```
x.shift(1).rolling_*
```

Không dùng:

```
x.rolling_*
```

Feature blocks và các đặc trưng phái sinh

Tất cả non-calendar weekly features bên dưới chỉ nên đưa vào Level 1 dưới dạng **lagged/rolling**.

1. Demand block từ `web_traffic.csv`

$$\text{sessions}_w = \sum \text{sessions}_d$$

$$\text{unique_visitors}_w = \sum \text{unique_visitors}_d$$

$$\text{page_views}_w = \sum \text{page_views}_d$$

$$\text{bounce_rate}_w = \text{weighted_mean}(\text{bounce_rate}_d, \text{sessions}_d)$$

$$\text{avg_session_duration}_w = \text{weighted_mean}(\text{duration}_d, \text{sessions}_d)$$

$$\text{pageviews_per_session}_w = \text{safe_div}(\text{page_views}_w, \text{sessions}_w)$$

Bổ sung thêm:

- `share_source_*`
- `source_entropy_w`

2. Conversion block từ `orders.csv`

$$\text{orders_count}_w = \sum \text{orders}_d$$

$$\text{conversion_proxy}_w = \text{safe_div}(\text{orders_count}_w, \text{sessions}_w)$$

$$\text{cancel_rate}_w = \text{safe_div}(\text{cancelled_orders}_w, \text{orders_count}_w)$$

$$\text{fulfilled_rate}_w = \text{safe_div}(\text{fulfilled_orders}_w, \text{orders_count}_w)$$

3. Basket block từ `sales.csv`, `order_items.csv`, `promotions.csv`

$$\text{AOV}_w = \text{safe_div}(\text{revenue}_w, \text{orders_count}_w)$$

$$\text{items_per_order}_w = \text{safe_div}(\text{units}_w, \text{orders_count}_w)$$

$$\text{discount_rate}_w = \text{safe_div}(\text{discount_amount}_w, \text{gross_before_discount}_w)$$

$$\text{promo_intensity}_w = \text{safe_div}(\text{promo_orders}_w, \text{orders_count}_w)$$

Nếu có COGS, có thể thêm:

$$\text{margin_rate}_w = \text{safe_div}(\text{revenue}_w - \text{cogs}_w, \text{revenue}_w)$$

4. Supply block từ `inventory.csv`, `shipments.csv`

$$\text{stockout_rate}_w = \text{safe_div}(\text{stockout_sku_days}_w, \text{active_sku_days}_w)$$

$$\text{fill_rate}_w = \text{safe_div}(\text{fulfilled_units}_w, \text{ordered_units}_w)$$

$$\text{days_of_supply}_w = \text{safe_div}(\text{on_hand_units}_w, \text{avg_daily_units_sold}_w)$$

$$\text{incoming_shipments}_w = \sum \text{received_units}_d$$

Nếu có inventory value:

$$\text{inventory_turnover}_{13w} = \text{safe_div}(\text{COGS}_{13w}, \text{avg_inventory_value}_{13w})$$

5. Leakage và quality block từ `returns.csv`, `reviews.csv`

$$\text{return_rate}_w = \text{safe_div}(\text{returned_orders}_w, \text{delivered_orders}_w)$$

$$\text{refund_rate}_w = \text{safe_div}(\text{refund_amount}_w, \text{revenue}_w)$$

$$\text{defective_ratio}_w = \text{safe_div}(\text{defective_returns}_w, \text{returned_orders}_w)$$

$$\text{wrong_size_ratio}_w = \text{safe_div}(\text{wrong_size_returns}_w, \text{returned_orders}_w)$$

$$\text{avg_rating}_w = \text{mean}(\text{rating})$$

$$\text{low_rating_share}_w = \text{safe_div}(\#\{\text{rating} \leq 2\}_w, \text{review_count}_w)$$

6. Customer block từ `customers.csv`, `orders.csv`

$$\text{active_customers}_w = \#\text{unique customers in week } w$$

$$\text{new_customer_ratio}_w = \text{safe_div}(\text{new_customers}_w, \text{active_customers}_w)$$

$$\text{repeat_customer_ratio}_w = \text{safe_div}(\text{repeat_customers}_w, \text{active_customers}_w)$$

$$\text{orders_per_customer}_w = \text{safe_div}(\text{orders_count}_w, \text{active_customers}_w)$$

7. Calendar block từ ngày

- `iso_week`, `month`, `quarter`
- `week_sin`, `week_cos` nếu muốn seasonality năm mượt hơn

- `is_month_start`, `is_month_end`
- `is_payday_window`
- `days_until_next_event`, `days_since_last_event`
- `optional promo_day_flag_known_ahead`

Với holiday handling dưới ràng buộc “không dùng dữ liệu ngoài”, cách an toàn nhất là:

- Feature lõi lấy từ logic ngày tháng xác định được.
- Event flag chỉ dùng nếu đã có sẵn trong `promotions.csv` hoặc lịch nội bộ được phê duyệt.
- Nếu không được phép dùng bảng public holiday bên ngoài, không import bảng đó.

Proxy lạm phát nội bộ

Không deflate doanh thu danh nghĩa bằng CPI. Thay vào đó, để mô hình học price drift danh nghĩa bằng proxy nội bộ:

- `AOV_lag_1w`
- `AOV_same_iso_week_1y`
- `AOV_yoy_hist`
- `COGS_per_order_lag`
- `revenue_same_iso_week_1y`
- `revenue_yoy_hist`
- `time_index_week`

Cách này thay thế logic “annual growth” thô trong baseline trend model bằng một cấu trúc tốt hơn: mô hình nhìn riêng được thay đổi về volume, thay đổi về price/mix và xu hướng dài hạn.

COVID smoothing chỉ dùng cho lag features

Tạo bản sao đã smoothing cho một vài chuỗi nhạy với long-lag:

- `revenue_smooth_for_lag`
- `sessions_smooth_for_lag`
- `orders_smooth_for_lag`
- `AOV_smooth_for_lag`

Không smoothing target dùng để huấn luyện và đánh giá. Chỉ smoothing bản sao dùng để tạo seasonal references dài hạn.

Quy tắc khuyến nghị:

- COVID window ở weekly grain: từ `2020-03-02` đến `2021-06-27`.
- Reference baseline cho mỗi `iso_week`: median của cùng ISO week trong giai đoạn 2018–2019.
- Winsor bounds:

$$\text{lower}_w = 0.5 \times \text{ref_median}_{iso_week}$$

$$\text{upper}_w = 1.5 \times \text{ref_median}_{iso_week}$$

Trong COVID window:

$$x_w^{\text{smooth}} = \text{clip}(x_w, \text{lower}_w, \text{upper}_w)$$

Chỉ dùng bản smoothed copy cho:

- `same_iso_week_last_year`
- fallback `lag_52w` / `lag_53w`
- `ma_13w` / `ma_52w` nếu tạo thêm

Các short lag như `lag_1w` và `lag_4w` vẫn dùng chuỗi raw.

Cold-start và quy tắc missing/zero

Cold-start phải được xử lý rõ ràng.

- Nếu chỉ dùng `lag_4w`, ta mất 4 tuần hoàn chỉnh đầu tiên.
- Nếu dùng annual reference và YoY proxy, thực tế mất khoảng 53 tuần hoàn chỉnh đầu tiên.
- Với Level 1 final training, loại bỏ các dòng thiếu P0 lag hoặc annual reference.
- Với Level 2 training, loại bỏ tuần có `revenue_week <= eps` vì khi đó daily weights không xác định.
- Với Level 3, chỉ bắt đầu từ validation window đầu tiên có OOF base daily predictions.

Chính sách null khuyến nghị:

- Count và amount bị thiếu vì không có sự kiện xảy ra: fill bằng 0.
- Share/rate có mẫu số bằng 0 và tử số bằng 0: fill bằng 0.
- Share/rate có mẫu số bằng 0 nhưng tử số khác 0: set null, sau đó impute bằng median của train-fold.
- Giá trị infinite: chuyển thành null trước khi impute.
- Growth feature cực đoan: clip theo percentile 1/99 của train-fold hoặc fixed bounds như `[-3, 3]`.

Các đoạn Polars mẫu

```
import polars as pl
from datetime import date

EPS = 1e-9

def build_daily_spine(start_dt: date, end_dt: date) -> pl.LazyFrame:
    return (
        pl.DataFrame({"date": pl.date_range(start_dt, end_dt, interval="1d",
```

```
eager=True)})
        .lazy()
        .with_columns(
            pl.col("date").dt.iso_year().alias("iso_year"),
            pl.col("date").dt.week().alias("iso_week"),
            pl.col("date").dt.weekday().alias("iso_weekday"),   # Monday=1,
Sunday=7
            pl.col("date").dt.month().alias("month"),
            pl.col("date").dt.quarter().alias("quarter"),
            pl.col("date").dt.day().alias("day_of_month"),
            pl.col("date").dt.ordinal_day().alias("day_of_year"),
        )
        .with_row_count("time_index_day")
        .with_columns(
            (pl.col("iso_year").cast(pl.Utf8) + "-W" +
             pl.col("iso_week").cast(pl.Utf8).str.zfill(2)).alias("week_id")
        )
    )
```

```
def weekly_aggregate(daily: pl.LazyFrame) -> pl.LazyFrame:
    return (
        daily.sort("date")
        .group_by_dynamic(
            "date",
            every="1w",
            period="1w",
            label="left",
            closed="left",
            start_by="monday",
        )
        .agg(
            pl.col("revenue").sum().alias("revenue_w"),
            pl.col("cogs").sum().alias("cogs_w"),
            pl.col("sessions").sum().alias("sessions_w"),
            pl.col("unique_visitors").sum().alias("unique_visitors_w"),
            pl.col("page_views").sum().alias("page_views_w"),

            # weighted means
            (pl.col("bounce_rate") * pl.col("sessions")).sum()
            / (pl.col("sessions").sum() + EPS)
            .alias("bounce_rate_w"),

            (pl.col("avg_session_duration_sec") * pl.col("sessions")).sum()
            / (pl.col("sessions").sum() + EPS)
            .alias("avg_session_duration_w"),
```

```
            pl.col("orders_count").sum().alias("orders_count_w"),
            pl.col("returned_orders").sum().alias("returned_orders_w"),
            pl.col("fulfilled_units").sum().alias("fulfilled_units_w"),
            pl.col("ordered_units").sum().alias("ordered_units_w"),
        )
        .rename({"date": "week_start"})
        .with_columns(
            pl.col("week_start").dt.iso_year().alias("iso_year"),
            pl.col("week_start").dt.week().alias("iso_week"),
        )
        .with_columns(
            (pl.col("page_views_w") / (pl.col("sessions_w") +
EPS)).alias("pageviews_per_session_w"),
            (pl.col("revenue_w") / (pl.col("orders_count_w") +
EPS)).alias("AOV_w"),
            (pl.col("fulfilled_units_w") / (pl.col("ordered_units_w") +
EPS)).alias("fill_rate_w"),
        )
        .with_columns(
            (pl.col("iso_year").cast(pl.Utf8) + "-W" +
             pl.col("iso_week").cast(pl.Utf8).str.zfill(2)).alias("week_id")
        )
    )
```

```
def add_past_only_lags(weekly: pl.LazyFrame) -> pl.LazyFrame:
    return (
        weekly.sort("week_start")
        .with_columns(
            pl.col("revenue_w").shift(1).alias("revenue_lag_1w"),
            pl.col("revenue_w").shift(4).alias("revenue_lag_4w"),
            pl.col("sessions_w").shift(1).alias("sessions_lag_1w"),
            pl.col("bounce_rate_w").shift(1).alias("bounce_rate_lag_1w"),
            pl.col("AOV_w").shift(1).alias("AOV_lag_1w"),

            # IMPORTANT: shift first, then roll

pl.col("sessions_w").shift(1).rolling_mean(4).alias("sessions_ma_4w"),

pl.col("sessions_w").shift(1).rolling_std(4).alias("sessions_vol_4w"),

            (
                (pl.col("sessions_w").shift(1) - pl.col("sessions_w").shift(2))
                / (pl.col("sessions_w").shift(2).abs() + EPS)
            ).clip(-3, 3).alias("sessions_growth_1w"),
        )
    )
```

Danh sách feature ưu tiên

Priority	Feature	Type	Source file(s)	Transform	Purpose
P0	`revenue_lag_1w`	weekly	`sales.csv`	weekly sum → lag 1	neo tự hồi quy mạnh nhất
P0	`revenue_lag_4w`	weekly	`sales.csv`	weekly sum → lag 4	giữ nhịp seasonality ngắn hạn
P0	`revenue_same_iso_week_1y_smooth`	weekly	`sales.csv`	self-join cùng ISO week trên bản smoothed copy	neo seasonality năm, tránh lệch 52/53 tuần
P0	`revenue_yoy_hist`	weekly	`sales.csv`	lagged YoY ratio từ historical weeks	proxy drift danh nghĩa nội bộ
P0	`time_index_week`	weekly	date	row index theo tuần	xu hướng dài hạn / proxy lạm phát
P0	`sessions_lag_1w`	weekly	`web_traffic.csv`	weekly sum → lag 1	quy mô nhu cầu gần nhất
P0	`sessions_ma_4w`	weekly	`web_traffic.csv`	shift(1) rolling mean 4	xu hướng nhu cầu ổn định
P0	`bounce_rate_lag_1w`	weekly	`web_traffic.csv`	weighted weekly mean → lag 1	chất lượng nhu cầu
P0	`source_entropy_lag_1w`	weekly	`web_traffic.csv`	source shares → entropy → lag 1	rủi ro lệ thuộc kênh
P0	`orders_count_lag_1w`	weekly	`orders.csv`	weekly count → lag 1	volume conversion gần nhất

Priority	Feature	Type	Source file(s)	Transform	Purpose
P0	`conversion_proxy_lag_1w`	weekly	`orders.csv` + `web_traffic.csv`	orders / sessions → lag 1	hiệu quả chuyển đổi demand-to-order
P0	`AOV_lag_1w`	weekly	`sales.csv` + `orders.csv`	revenue / orders → lag 1	tín hiệu price/mix
P0	`AOV_yoy_hist`	weekly	`sales.csv` + `orders.csv`	lagged YoY ratio	proxy lạm phát nội bộ
P0	`stockout_rate_lag_1w`	weekly	`inventory.csv`	stockout sku-days / active sku-days → lag 1	nhận diện giới hạn nguồn cung
P0	`fill_rate_lag_1w`	weekly	`inventory.csv` / `shipments.csv`	fulfilled / ordered units → lag 1	khả năng thực thi supply
P0	`return_rate_lag_1w`	weekly	`returns.csv`	returned / delivered orders → lag 1	penalty revenue leakage
P0	`defective_ratio_lag_1w`	weekly	`returns.csv`	defective / all returns → lag 1	tín hiệu sốc chất lượng
P0	`repeat_customer_ratio_lag_1w`	weekly	`customers.csv` + `orders.csv`	repeat / active customers → lag 1	chất lượng khách hàng
P0	`iso_week`, `month`	weekly	date	known-ahead calendar	seasonality
P1	`avg_rating_lag_1w`	weekly	`reviews.csv`	weekly mean → lag 1	sentiment khách hàng
P1	`discount_rate_lag_1w`	weekly	`promotions.csv` / `order_items.csv`	discount / gross → lag 1	cường độ khuyến mãi

Priority	Feature	Type	Source file(s)	Transform	Purpose
P1	`incoming_shipments_lag_1w`	weekly	`shipments.csv`	weekly received units → lag 1	tín hiệu bổ sung hàng
P1	`sessions_vol_4w`	weekly	`web_traffic.csv`	shift(1) rolling std 4	bất ổn nhu cầu
P0	`weekday`	daily	date	known-ahead categorical	hình dạng ngày mặc định
P0	`day_of_month`	daily	date	known-ahead integer	vị trí trong tháng
P0	`month`	daily	date	known-ahead categorical	hình dạng seasonal theo ngày
P0	`days_until_next_event`	daily	date + allowed event calendar	event lead distance	lift trước sự kiện
P0	`days_since_last_event`	daily	date + allowed event calendar	event lag distance	decay sau sự kiện
P0	`is_payday_window`	daily	date	deterministic payday bucket	hiệu ứng chu kỳ lương
P0	`fallback_weight_hist`	daily	training history	grouped historical mean weight	allocator fallback an toàn
P1	`promo_day_flag_known_ahead`	daily	`promotions.csv`	future-known binary nếu được phép	allocation có nhận biết event

Weekly forecasting core

Target của Level 1 là doanh thu danh nghĩa theo tuần:

$$y_t^{(w)} = \log(1 + \mathrm{RevenueWeek}_t)$$

Dự báo ngược về raw scale:

$$\widehat{\mathrm{RevenueWeek}}_t = \max(\exp(\hat{y}_t^{(w)}) - 1, 0)$$

Transform này phù hợp vì xử lý được giá trị 0 và bảo đảm weekly forecast không âm sau khi inverse.

Với weekly learner, mô hình chính nên là **LightGBM** trên các weekly features đã engineering. Vì lịch sử tuần từ 2012–2022 chỉ khoảng 570 dòng, cần giữ mô hình conservative.

Model Level 1 khuyến nghị

Primary model:

- Library: `lightgbm`
- Target: `log1p(revenue_week)`
- Objective: `regression_l1`
- Eval metrics: `l1`, `rmse`
- Early stopping: 200

Secondary challenger:

- Cùng feature và fold
- Objective: `huber`

Optional experiment:

- `gamma` hoặc `tweedie` trên raw weekly revenue nếu target tuần luôn dương và số 0 rất hiếm.

Khoảng hyperparameter Level 1

Parameter	Primary range	Notes
`learning_rate`	0.03–0.05	học chậm, ổn định hơn
`n_estimators`	1500–4000	dùng early stopping
`num_leaves`	15–31	nhỏ vì sample tuần thấp
`max_depth`	4–6	chống overfit
`min_data_in_leaf`	20–60	tăng nếu cây không ổn định
`feature_fraction`	0.7–0.9	subsampling theo cột
`bagging_fraction`	0.7–0.9	subsampling theo dòng
`bagging_freq`	1	bật bagging
`lambda_l1`	1–10	tăng sparsity / robustness
`lambda_l2`	5–30	ổn định hơn với sample nhỏ
`min_gain_to_split`	0–0.1	regularization thêm nếu cần

Training folds và OOF generation

Backtesting nên dùng **rolling forecasting origin**: mỗi validation block chỉ chứa dữ liệu tương lai so với training block. Đây là nguyên tắc chuẩn của time-series cross-validation.

Với dataset này, nên dùng các fold theo business year thay vì `TimeSeriesSplit` generic, vì validation period dễ diễn giải hơn.

Fold	Train complete ISO weeks	Validate complete ISO weeks	Ý nghĩa
F1	2012-W01 đến 2018-W52	2019-W01 đến 2019-W52	kiểm tra generalization trước shock
F2	2012-W01 đến 2019-W52	2020-W01 đến 2020-W53	kiểm tra robustness trong shock
F3	2012-W01 đến 2020-W53	2021-W01 đến 2021-W52	kiểm tra recovery regime
F4	2012-W01 đến 2021-W52	2022-W01 đến 2022-W52	rehearsal cuối trước forecast

Với mỗi fold:

1. Fit Level 1 trên train weeks.
2. Predict Level 1 trên validation weeks.
3. Lưu OOF weekly predictions.
4. Fit Level 2 allocator trên training days.
5. Predict daily validation weights.
6. Tạo OOF base daily forecasts.
7. Dùng OOF daily base để train Level 3 residual.

OOF chain là bắt buộc. Không train Level 3 trên base daily values được tạo từ full-sample weekly fit.

Pseudo-code weekly model

```
import numpy as np
import pandas as pd
import lightgbm as lgb

L1_FEATURES = [
    "revenue_lag_1w", "revenue_lag_4w", "revenue_same_iso_week_1y_smooth",
    "revenue_yoy_hist", "time_index_week",
    "sessions_lag_1w", "sessions_ma_4w", "bounce_rate_lag_1w",
    "orders_count_lag_1w", "conversion_proxy_lag_1w",
    "AOV_lag_1w", "AOV_yoy_hist",
    "stockout_rate_lag_1w", "fill_rate_lag_1w",
    "return_rate_lag_1w", "defective_ratio_lag_1w",
    "repeat_customer_ratio_lag_1w",
    "iso_week", "month"
```

```
]

L1_PARAMS = dict(
    objective="regression_l1",
    metric=["l1", "rmse"],
    learning_rate=0.03,
    num_leaves=23,
    max_depth=5,
    min_data_in_leaf=30,
    feature_fraction=0.8,
    bagging_fraction=0.8,
    bagging_freq=1,
    lambda_l1=3.0,
    lambda_l2=10.0,
    verbosity=-1,
)

def fit_level1(train_w, valid_w):
    y_tr = np.log1p(train_w["revenue_w"].values)
    y_va = np.log1p(valid_w["revenue_w"].values)

    dtr = lgb.Dataset(train_w[L1_FEATURES], label=y_tr)
    dva = lgb.Dataset(valid_w[L1_FEATURES], label=y_va, reference=dtr)

    model = lgb.train(
        L1_PARAMS,
        dtr,
        num_boost_round=4000,
        valid_sets=[dva],
        callbacks=[lgb.early_stopping(200, first_metric_only=True,
verbose=False)]
    )
    pred_log = model.predict(valid_w[L1_FEATURES],
num_iteration=model.best_iteration)
    pred_raw = np.maximum(np.expm1(pred_log), 0.0)
    return model, pred_raw
```

Rolling-origin backtest skeleton

```
def wape(y_true, y_pred, eps=1e-9):
    denom = np.abs(y_true).sum()
    if denom <= eps:
        return np.nan
    return np.abs(y_true - y_pred).sum() / denom

FOLDS = [
```

```
    ("2012-W01", "2018-W52", "2019-W01", "2019-W52"),
    ("2012-W01", "2019-W52", "2020-W01", "2020-W53"),
    ("2012-W01", "2020-W53", "2021-W01", "2021-W52"),
    ("2012-W01", "2021-W52", "2022-W01", "2022-W52"),
]

weekly_oof = []

for tr_start, tr_end, va_start, va_end in FOLDS:
    tr = weekly_df[(weekly_df.week_id >= tr_start) & (weekly_df.week_id <=
tr_end)].copy()
    va = weekly_df[(weekly_df.week_id >= va_start) & (weekly_df.week_id <=
va_end)].copy()

    model_l1, va["rev_week_pred_oof"] = fit_level1(tr, va)
    weekly_oof.append(va[["week_id", "week_start", "revenue_w",
"rev_week_pred_oof"]])

weekly_oof = pd.concat(weekly_oof, ignore_index=True)
```

Daily allocation, residual correction và reconciliation

Level 2: Daily allocation

Level 2 không dự báo trực tiếp doanh thu ngày. Nó dự báo **trọng số trong tuần**.

Với ngày d trong tuần w:

$$\text{weight}_{d,w} = \frac{\text{Revenue}_d}{\max(\text{RevenueWeek}_w, \varepsilon)}$$

Với ràng buộc:

$$\text{weight}_{d,w} \geq 0, \qquad \sum_{d \in w} \text{weight}_{d,w} = 1$$

Mẹo thực tế là dự báo một daily score không ràng buộc, sau đó chuyển score thành weight bằng softmax theo tuần:

$$\text{score}_{d,w} = f_{\text{alloc}}(X_{d,w})$$

$$\text{weight}_{d,w} = \frac{\exp(\text{score}_{d,w} - m_w)}{\sum_{j \in w} \exp(\text{score}_{j,w} - m_w)}$$

Trong đó:

$$m_w = \max_{j \in w} \text{score}_{j,w}$$

Softmax giúp bảo đảm:

- Không có trọng số âm.
- Tổng trọng số trong tuần bằng 1.
- Base daily forecast nhất quán.

Sau đó:

$$\text{BaseDaily}_{d,w} = \overline{\text{RevenueWeek}}_w \times \text{weight}_{d,w}$$

Feature được phép dùng cho Level 2

Chỉ dùng daily features biết trước:

- `weekday`
- `day_of_month`
- `month`
- `quarter`
- `is_month_start`, `is_month_end`
- `days_until_next_event`
- `days_since_last_event`
- `is_payday_window`
- `promo_day_flag_known_ahead` chỉ dùng nếu future promo schedule thật sự được cung cấp

Nếu lịch event nội bộ thưa hoặc không được phép dùng, pure calendar allocator với weekday/month/payday vẫn có thể hoạt động tốt.

Fallback rules

Fallback là bắt buộc.

Dùng fallback nếu xảy ra một trong các trường hợp:

- Tất cả score trong một tuần bị missing.
- Mẫu số softmax không hữu hạn hoặc gần 0.
- Weekly forecast gần bằng 0.
- Event signature chưa từng xuất hiện trong training.

Fallback weights:

$$\text{fallback_weight}_{d,w} = \mathbb{E}[\text{weight}_{d,w} \mid \text{weekday}, \text{month}, \text{event bucket}]$$

Back-off hierarchy:

1. `weekday + month + event_bucket`
2. `weekday + event_bucket`

3. `weekday only`
4. uniform `1/7`

Pseudo-code Level 2

```
import numpy as np
import pandas as pd

ALLOC_FEATURES = [
    "weekday", "day_of_month", "month",
    "days_until_next_event", "days_since_last_event",
    "is_payday_window"
]

def softmax_by_week(df, score_col="score", week_col="week_id"):
    out = df.copy()
    max_score = out.groupby(week_col)[score_col].transform("max")
    z = np.exp(out[score_col] - max_score)
    z_sum = out.groupby(week_col)[score_col].transform(lambda s: np.exp(s -
s.max()).sum())
    out["weight_pred"] = np.where(z_sum > 0, z / z_sum, np.nan)
    return out

def apply_fallback_weights(df, fallback_col="fallback_weight_hist"):
    out = df.copy()
    bad = out["weight_pred"].isna() | ~np.isfinite(out["weight_pred"])
    if bad.any():
        out.loc[bad, "weight_pred"] = out.loc[bad, fallback_col]
    # re-normalize inside each week after fallback
    out["weight_pred"] = out["weight_pred"].clip(lower=0.0)
    denom = out.groupby("week_id")["weight_pred"].transform("sum")
    out["weight_pred"] = np.where(denom > 0, out["weight_pred"] / denom, 1.0 /
7.0)
    return out
```

Level 3: Residual correction

Level 3 là tùy chọn và nên được xem là một ablation, không phải thành phần chắc chắn phải có.

Nhiệm vụ của Level 3 chỉ là sửa các lỗi daily misshape có tính hệ thống còn lại sau Level 1 + Level 2.

Dùng multiplicative residual target:

$$r_{d,w} = \log\left(\frac{\text{ActualDaily}_{d,w} + \varepsilon}{\text{BaseDaily}_{d,w} + \varepsilon}\right)$$

Sau đó:

$$\text{AdjustedDaily}_{d,w} = \text{BaseDaily}_{d,w} \times \exp(\hat{r}_{d,w})$$

Cách này giữ adjusted daily forecast không âm tự động.

Quy tắc quan trọng: chỉ train Level 3 trên **OOF base daily forecasts**, không train trên in-sample weekly fits.

Regularization khuyến nghị cho Level 3:

- Objective: `regression_l2` hoặc `huber`
- `max_depth = 3–4`
- `num_leaves = 7–15`
- `min_data_in_leaf = 100–300`
- `lambda_l1`, `lambda_l2` mạnh
- Clip residual target theo percentile 1/99 của train-fold hoặc fixed `[-2, 2]`

Loại bỏ Level 3 nếu:

- Mean daily WAPE không cải thiện ít nhất 1.5% relative.
- Mean daily MAE không cải thiện ít nhất 1.0% relative.
- Bất kỳ validation fold nào xấu đi đáng kể so với chỉ dùng Level 2.

Đây là quy tắc chống nhiễu phù hợp.

Pseudo-code Level 3 OOF

```
RESID_FEATURES = [
    "weekday", "day_of_month", "month",
    "days_until_next_event", "days_since_last_event",
    "is_payday_window"
]

def build_residual_target(df, actual_col="revenue", base_col="base_daily_oof",
eps=1e-9):
    r = np.log((df[actual_col].values + eps) / (df[base_col].values + eps))
    return np.clip(r, -2.0, 2.0)

# daily_oof must already contain:
# week_id, date, actual revenue, rev_week_pred_oof, weight_pred_oof
daily_oof["base_daily_oof"] = daily_oof["rev_week_pred_oof"] *
daily_oof["weight_pred_oof"]
daily_oof["r_target"] = build_residual_target(daily_oof)
```

```
# fit residual model only on OOF rows
# residual_model.fit(daily_oof[RESID_FEATURES], daily_oof["r_target"])
```

Level 4: Safe reconciliation

Sau residual correction, tổng daily values có thể không còn đúng bằng weekly forecast. Level 4 sửa vấn đề này.

Với mỗi tuần w:

$$S_w = \sum_{d \in w} \text{AdjustedDaily}_{d,w}$$

Nếu $S_w > \varepsilon$, reconcile theo tỷ lệ:

$$\text{FinalDaily}_{d,w} = \text{AdjustedDaily}_{d,w} \times \frac{\overline{\text{RevenueWeek}}_w}{S_w}$$

Nếu không, dùng fallback weights:

$$\text{FinalDaily}_{d,w} = \overline{\text{RevenueWeek}}_w \times \text{fallback_weight}_{d,w}$$

Đây là tầng chống crash. Nó ngăn divide-by-zero, NaN và weekly sums bị sai.

Nếu `RevenueWeek_pred <= eps`, đặt tất cả ngày trong tuần đó bằng 0 và bỏ qua Level 3/4 transforms.

Pseudo-code safe reconciliation

```
def safe_reconcile_one_week(g, eps=1e-9):
    g = g.copy()
    target = float(g["rev_week_pred"].iloc[0])

    if target <= eps:
        g["final_daily"] = 0.0
        return g

    adjusted = np.maximum(g["adjusted_daily"].to_numpy(), 0.0)
    s = adjusted.sum()

    if np.isfinite(s) and s > eps:
        g["final_daily"] = adjusted * (target / s)
    else:
        w = np.maximum(g["fallback_weight_hist"].to_numpy(), 0.0)
        w_sum = w.sum()
        if (not np.isfinite(w_sum)) or (w_sum <= eps):
```

```
            w = np.repeat(1.0 / len(g), len(g))
        else:
            w = w / w_sum
        g["final_daily"] = target * w

    return g

final_daily = (
    adjusted_daily_df
    .groupby("week_id", group_keys=False)
    .apply(safe_reconcile_one_week)
)
```

Với các tuần đầy đủ, reconciliation drift gần như bằng 0, ngoại trừ sai số floating-point rất nhỏ. Nếu sau này cần làm tròn đến đơn vị tiền tệ, hãy làm tròn sau Level 4 rồi đẩy phần sai lệch do rounding vào ngày có weight cao nhất trong tuần.

Validation, release gates và vận hành

Đánh giá cần thực hiện ở cả hai cấp:

- Level 1 weekly metrics trên weekly holdout.
- Final daily metrics trên final daily holdout.

Dùng bốn metric:

$$\mathrm{MAE} = \frac{1}{n}\sum |y - \hat{y}|$$

$$\mathrm{RMSE} = \sqrt{\frac{1}{n}\sum (y - \hat{y})^2}$$

$$R^2 = 1 - \frac{\sum (y - \hat{y})^2}{\sum (y - \bar{y})^2}$$

$$\mathrm{WAPE} = \frac{\sum |y - \hat{y}|}{\sum |y|}$$

MAE và RMSE là loss không âm, giá trị tốt nhất là 0. R^2 có thể âm nếu mô hình tệ hơn việc dự báo bằng trung bình. WAPE hữu ích cho giao tiếp kinh doanh, nhưng không xác định khi tổng actual bằng 0, nên cần dùng cùng MAE/RMSE/R^2 thay vì dùng một mình.

Promotion thresholds

Giữ notebook hiện tại làm **Benchmark A**.

Chỉ promote Level 1 + Level 2 + Level 4 sang giai đoạn tiếp theo nếu tất cả điều kiện sau đúng:

- Mean daily WAPE cải thiện ít nhất 10% relative so với Benchmark A trên F1–F4.
- Mean daily MAE cải thiện ít nhất 5% relative.
- Mean daily RMSE không xấu đi.
- Không fold nào xấu hơn Benchmark A quá 3% relative WAPE.
- Số daily prediction âm = 0.
- Weekly coherence drift sau Level 4 < 0.1% ở p99 và gần 0 ở trung bình.
- P0 feature null rate sau preprocessing < 0.5%.

Chỉ bật Level 3 nếu:

- Cải thiện mean daily WAPE $\geq$ 1.5% relative so với chỉ dùng Level 2.
- Cải thiện mean daily MAE $\geq$ 1.0% relative.
- Không làm xấu đáng kể bất kỳ fold nào.
- Residual mean trên holdout vẫn gần 0.

Monitoring KPIs

Theo dõi ba nhóm tín hiệu.

1. Data-quality KPIs

- Row count theo source và theo ngày.
- Freshness lag theo source.
- P0 feature null rate.
- Non-finite ratio count.
- Share of days missing from spine joins.

2. Forecast-integrity KPIs

$$\text{coherence drift}_w = \frac{\left|\sum_{d \in w} \text{FinalDaily}_{d,w} - \widehat{\text{RevenueWeek}}_w\right|}{\max(\widehat{\text{RevenueWeek}}_w, \varepsilon)}$$

Theo dõi thêm:

- Negative prediction count.
- Zero-sum week count.
- Fallback-rate trong Level 2 và Level 4.

3. Business-risk KPIs

Stockout warning:

- `stockout_rate_lag_1w` vượt historical p95 trong khi weekly forecast vẫn cao.

Return-quality warning:

- `return_rate_lag_1w` hoặc `defective_ratio_lag_1w` tăng đột biến.

Residual distribution drift:

- Trailing-8-week residual mean nằm ngoài `[-0.05, 0.05]`.
- Residual std vượt 1.5 lần training std.

Weekday allocation drift:

- Current weekday shares lệch hơn 5 percentage points so với pattern trailing 26-week.

Deployment checklist

Category	Job	Cadence	Success condition	Alert trigger
Data job	Ingest raw CSVs to bronze	once per batch / run	đủ file kỳ vọng	thiếu file hoặc schema mismatch
Data job	Build daily calendar spine	once per batch / run	đủ date coverage cho train + forecast horizon	thiếu ngày
Data job	Build daily marts by block	once per batch / run	một dòng mỗi ngày mỗi mart	duplicate dates hoặc row explosion
Data job	Join daily marts to feature mart	once per batch / run	P0 null rate < 0.5%	null tăng đột biến
Data job	Aggregate weekly mart	once per batch / run	chỉ dùng complete weeks	nhiễm partial-week
Data job	Persist marts to Parquet	once per batch / run	artifact có version được ghi	thiếu artifact
Model job	Train Level 1 weekly model	per retrain cycle	fold metrics được log; early stopping hội tụ	fold fail / overfit nặng
Model job	Train Level 2 allocator	per retrain cycle	fallback table và model được lưu	softmax bất ổn
Model job	Train Level 3 residual	only if gate passes	tốt hơn Level 2 trên OOF eval	không có uplift
Model job	Generate OOF predictions	per retrain cycle	OOF coverage đủ trên mọi fold	thiếu OOF
Model job	Batch-predict future weeks	forecast run	tất cả future complete weeks được dự báo	thiếu future weeks

Category	Job	Cadence	Success condition	Alert trigger
Model job	Allocate / correct / reconcile daily	forecast run	không có giá trị âm; coherence drift gần 0	divide-by-zero / NaN / fallback quá nhiều
Model job	Export final daily forecast	forecast run	row count khớp required horizon	submission thiếu dòng
Monitoring	Track weekly coherence drift	every forecast run	p99 < 0.1%	drift vượt ngưỡng
Monitoring	Track stockout warning rate	weekly	trong biên bình thường	vượt p95
Monitoring	Track residual mean/std	after actuals arrive	mean gần 0, std ổn định	bias drift
Monitoring	Track data freshness and nulls	every batch	trong SLA	upstream stale hoặc lỗi

Development timeline

```
timeline
    title Delivery roadmap
    Baseline freeze : Keep current notebook as Benchmark A
    Data foundation : Build daily spine
                    : Build block-level daily marts
                    : Join marts and save Parquet artifacts
    Weekly core : Add ISO-week logic
                : Add COVID lag smoothing
                : Add lag / rolling / YoY features
                : Train Level 1 and run rolling-origin backtests
    Allocation layer : Start with historical weekday weights
                     : Upgrade to softmax allocator
                     : Add fallback hierarchy
    Residual layer : Build OOF weekly and daily base predictions
                   : Train tightly regularized residual model
                   : Keep only if validation improves
    Hardening : Add safe reconciliation
              : Add boundary-day shim
              : Add monitoring and release gates
```

Thứ tự triển khai cuối cùng

Trình tự triển khai sạch nhất:

1. Đóng băng notebook hiện tại làm benchmark.
2. Xây dựng daily spine và block marts bằng Polars.
3. Xây dựng weekly mart.
4. Thêm annual ISO-week references và COVID-smoothed lag copies.
5. Train Level 1 weekly model và validate trên F1–F4.
6. Thêm Level 2 bằng historical weekday weights trước.
7. Thay historical weights bằng ML softmax allocation.
8. Sinh OOF base daily forecasts.
9. Chỉ thêm Level 3 nếu vượt release gate.
10. Hoàn thiện Level 4 reconciliation và monitoring.

Kết luận: đây là phương án cân bằng tốt nhất giữa **độ chính xác**, **an toàn chống leakage**, **độ bền vận hành** và **độ rõ ràng khi triển khai** cho bài toán dự báo doanh thu danh nghĩa theo ngày giai đoạn 2023–2024 từ các file dữ liệu 2012–2022 được cung cấp.